



Fullstack Web Development

Javascript - Part 02

Instructor: Ms. Liev Nova



090667393



Class

A **class** is a blueprint used to create objects.

It represents a **real-world object**

Example:

- Dog
- Studebt
- Car

Each has:

- **Properties (fields)** → data
- **Methods** → actions

Class Syntax

```
class ClassName {
  constructor() {
    // initialize properties
  }

  methodName() {
    // method code
  }
}
```

Class

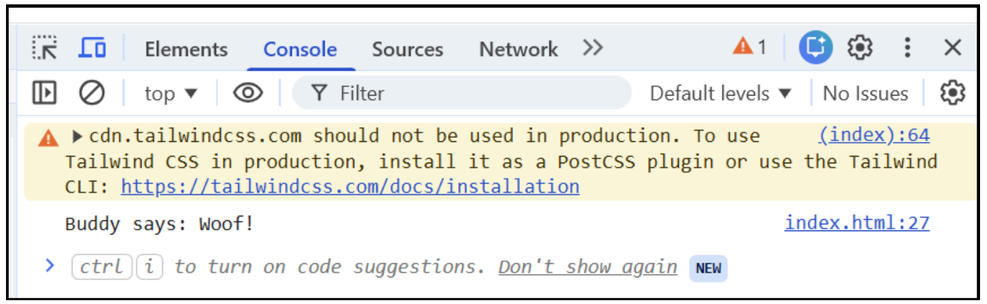
Basic Class Example:

```
class Dog {
    constructor(name) {
      this.name = name;
    }

    bark() {
      console.log(this.name + " says: Woof!");
    }
  }

  // Create object
  let dog1 = new Dog("Buddy");

  dog1.bark();
```



- class : blueprint
- constructor : runs when object is created
- this : refers to current object
- new : creates object

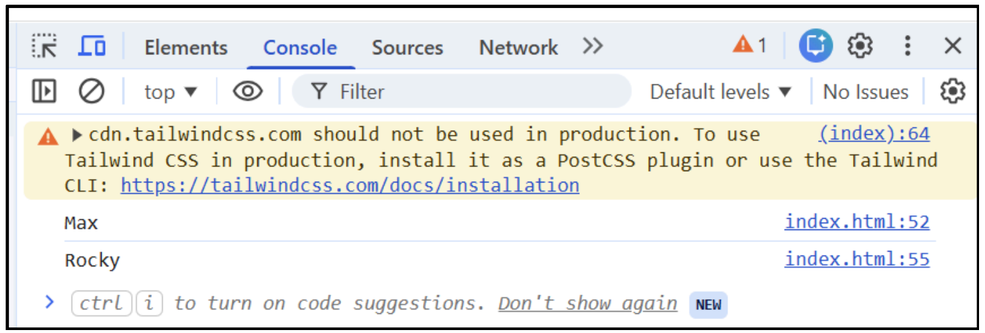
Class

Getter and Setter

What is Getter & Setter?

- get → retrieve value
- set → update value

Example:




```
class Dog {
  constructor(name) {
    this._name = name; // use different variable
  }

  get name() {
    return this._name;
  }

  set name(newName) {
    this._name = newName;
  }
}

let dog = new Dog("Max");

console.log(dog.name); // getter

dog.name = "Rocky";    // setter
console.log(dog.name);
```

Important Rule : Always use a different property (like _name) inside getter/setter

Class

Inheritance

What is Inheritance?

A class can **inherit properties and methods from another class**

Why use it?

- Code reuse
- Cleaner structure
- Avoid repetition

Syntax:

```
class ChildClass extends ParentClass {
  constructor() {
    super(); // call parent constructor
  }
}
```

Class

Inheritance

- Example

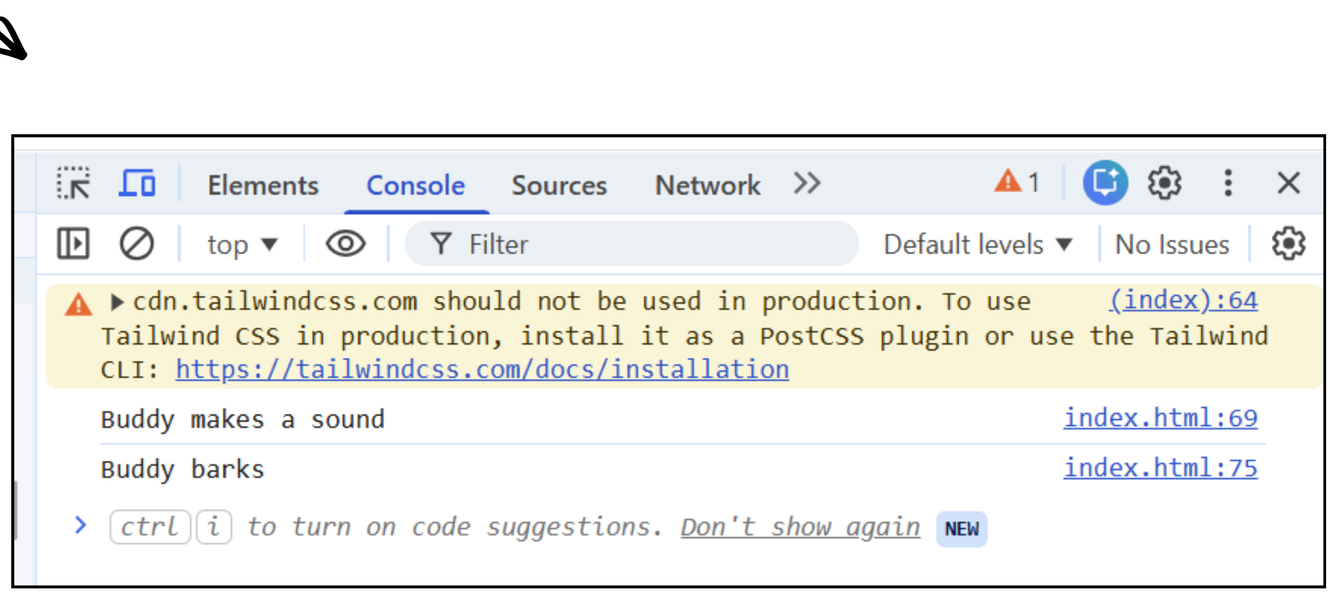
```
class Animal {
  constructor(name) {
    this.name = name;
  }

  speak() {
    console.log(this.name + " makes a sound");
  }
}

class Dog extends Animal {
  bark() {
    console.log(this.name + " barks");
  }
}

let dog = new Dog("Buddy");

dog.speak();
dog.bark();
```



Class

super() Keyword

What is super()?

super() : Calls the **parent class constructor**

Example:

```
class Dog extends Animal {
  constructor(name) {
    super(name); // pass value to parent
  }
}
```

ASYNC

What is Asynchronous JavaScript?

Asynchronous code allows JavaScript to **run tasks without blocking other code**

Example:

- Waiting for data
- Timer
- API request

Synchronous = step by step
Asynchronous = can run later (non-blocking)

ASYNC

What is Asynchronous JavaScript?

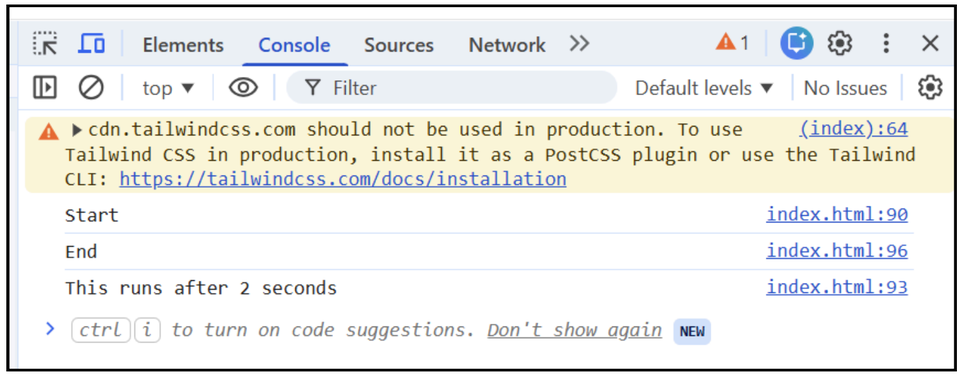
Asynchronous code allows JavaScript to **run tasks without blocking other code**

Example: (setTimeout)

```
console.log("Start");

setTimeout(function () {
  console.log("This runs after 2 seconds");
}, 2000);

console.log("End");
```



Important: setTimeout runs later, not immediately

ASYNC

Callback Function vs Asynchronous

Callback Function : a function passed into another function and executed later
It can be **synchronous OR asynchronous**

Asynchronous : Code runs **later**, without blocking other code

Callback = "What to do next"
Async = "Do it later"

Callback ≠ Async , BUT... Async often **uses callbacks**

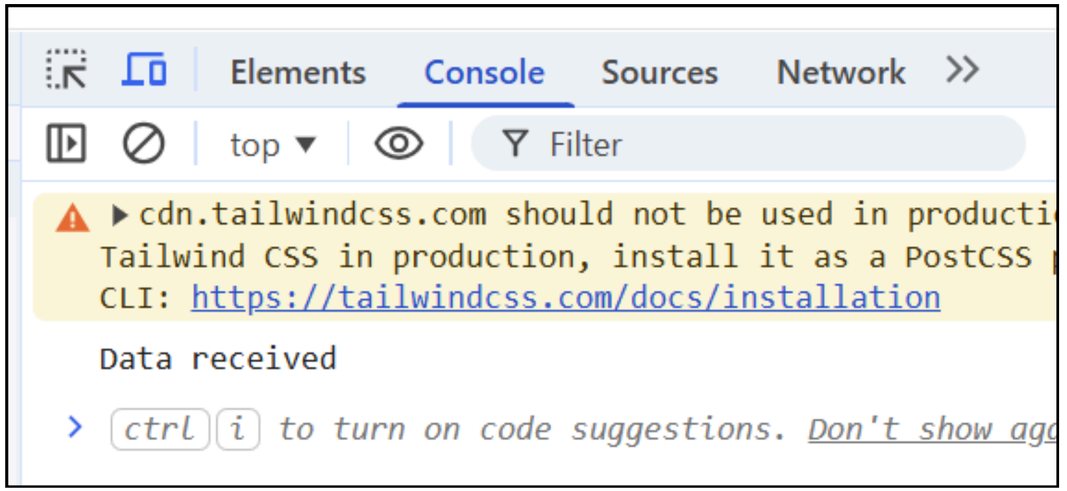
ASYNC

Callback Function vs Asynchronous

Example: Callback + Async Together

```
function fetchData(callback) {
  setTimeout(() => {
    callback("Data received");
  }, 2000);
}

fetchData(function (result) {
  console.log(result);
});
```



ASYNC

Problem with Callback (Callback Hell)

```
setTimeout(() => {
  console.log("Step 1");

  setTimeout(() => {
    console.log("Step 2");

    setTimeout(() => {
      console.log("Step 3");
    }, 1000);

  }, 1000);

}, 1000);
```

❗ Hard to read → called Callback Hell

Promise in JavaScript

What is Promise?

A **Promise** is an object that represents a **future result.**

Example: "I will give you the result later"

It connects:

- Producing code (takes time)
- Consuming code (waits for result)

Promise in JavaScript

Producing vs Consuming

Producing Code — Code that takes time (async)

Examples:

- Loading data
- Waiting (setTimeout)
- API request

Consuming Code — Code that waits for the result

Uses:

```
.then()
.catch()
```

```
promise
  .then(result => {
    console.log("Success:", result);
  })
  .catch(error => {
    console.log("Error:", error);
  });
```

Promise in JavaScript

Promise States:

State	Meaning
pending	waiting
fulfilled	success
rejected	error

Basic Syntax:

```
let promise = new Promise((resolve, reject) => {
  // producing code
});
```

Important Functions:

- resolve(value) → success
- reject(error) → error

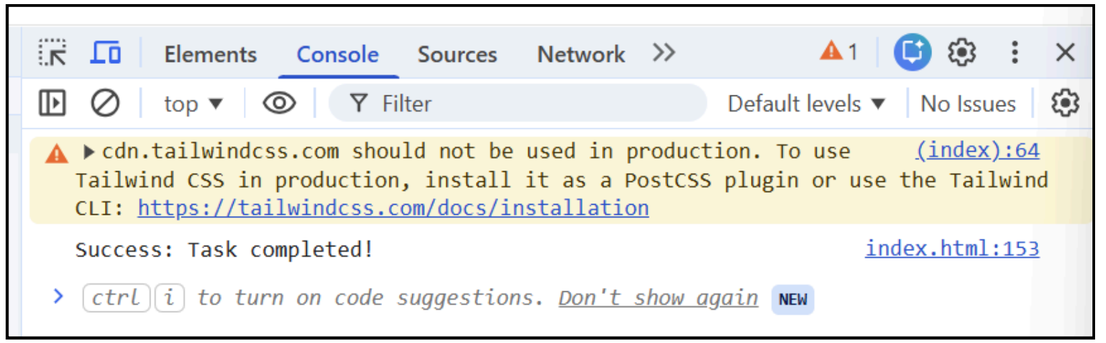
Promise in JavaScript

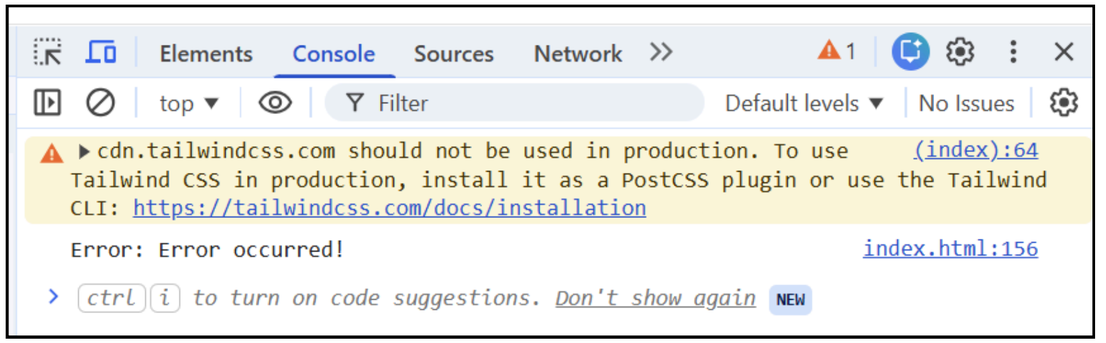
Promise Example:

```
let myPromise = new Promise(function (resolve, reject) {
  let success = true;

  if (success) {
    resolve("Task completed!");
  } else {
    reject("Error occurred!");
  }
});

myPromise.then(
  function (result) {
    console.log("Success:", result);
  },
  function (error) {
    console.log("Error:", error);
  }
);
```



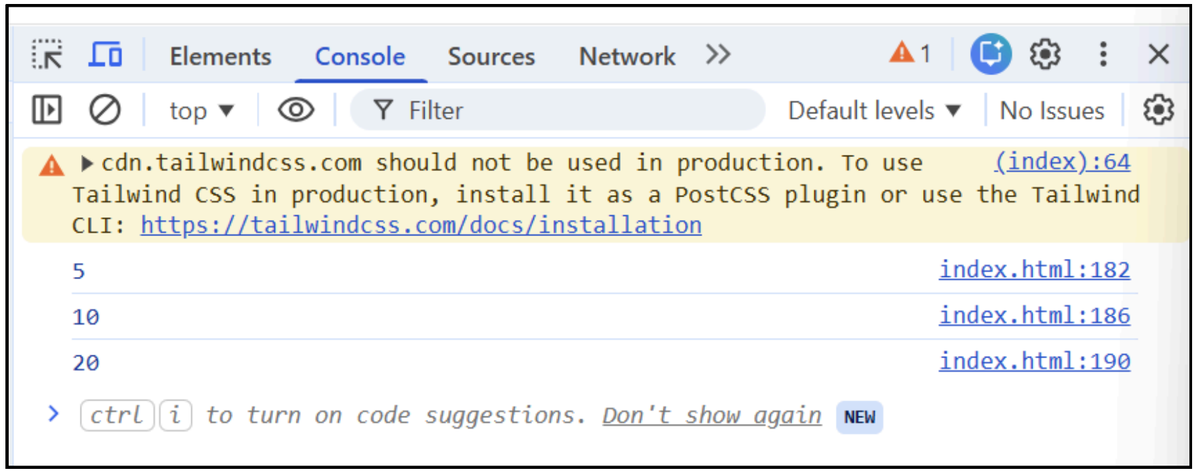


Promise in JavaScript

Promise Chaining Run — multiple async steps

```
new Promise((resolve, reject) => {
  resolve(5);
})
  .then(num => {
    console.log(num);
    return num * 2;
  })
  .then(num => {
    console.log(num);
    return num * 2;
  })
  .then(num => {
    console.log(num);
  });
```



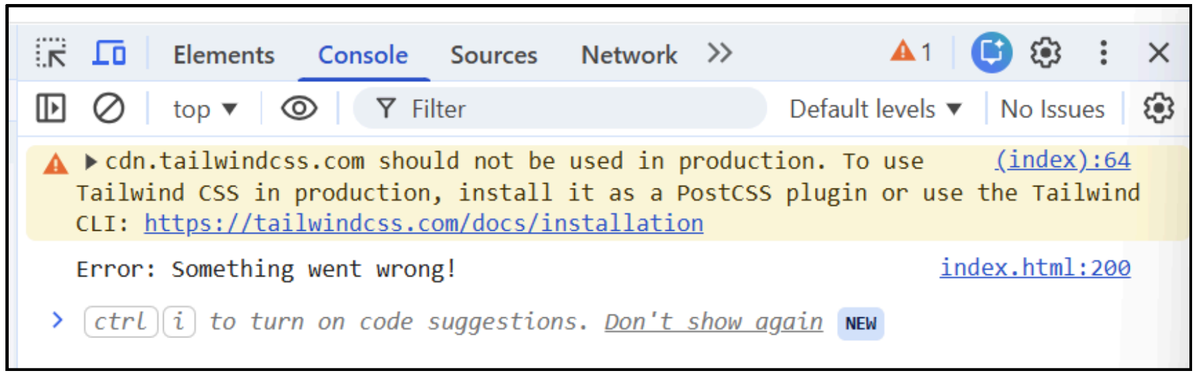


Promise in JavaScript

Promise Error Handling

```
new Promise((resolve, reject) => {
  reject("Something went wrong!");
})
  .then(result => {
    console.log(result);
  })
  .catch(error => {
    console.log("Error:", error);
  });
```



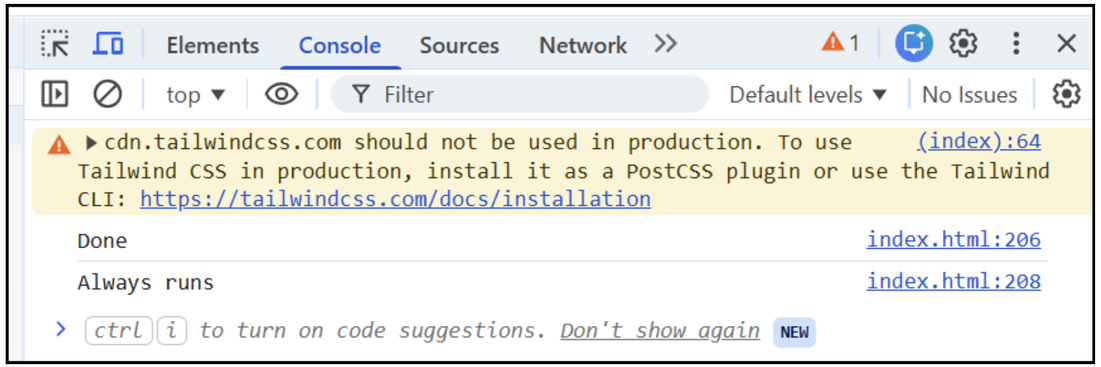


Promise in JavaScript

Promise — finally() Runs no matter what

```
new Promise((resolve, reject) => {
  resolve("Done");
})
  .then(result => console.log(result))
  .catch(error => console.log(error))
  .finally(() => console.log("Always runs"));
```





Promise in JavaScript

Promise vs Callback

```
new Promise(resolve => {
  setTimeout(resolve, 2000);
}).then(() => console.log("Done"));
```

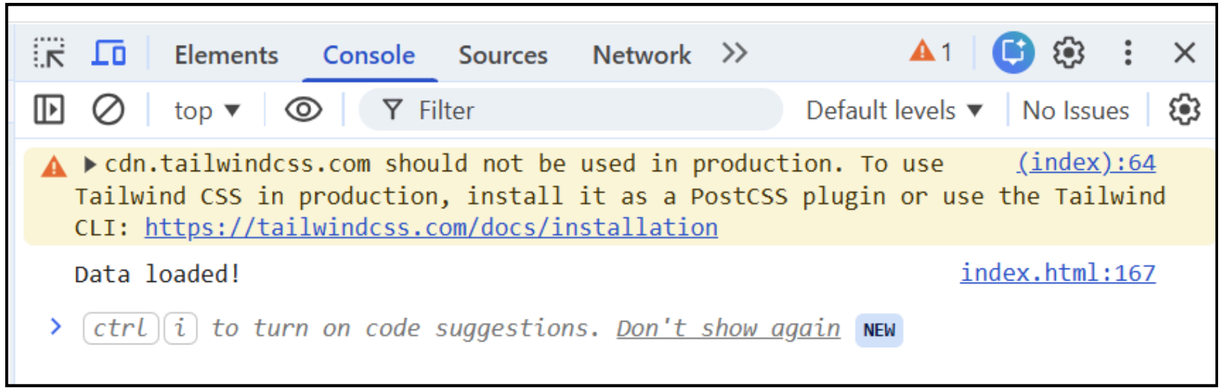
```
setTimeout(() => {
  console.log("Done");
}, 2000);
```

Promise in JavaScript

Promise with setTimeout (Async)

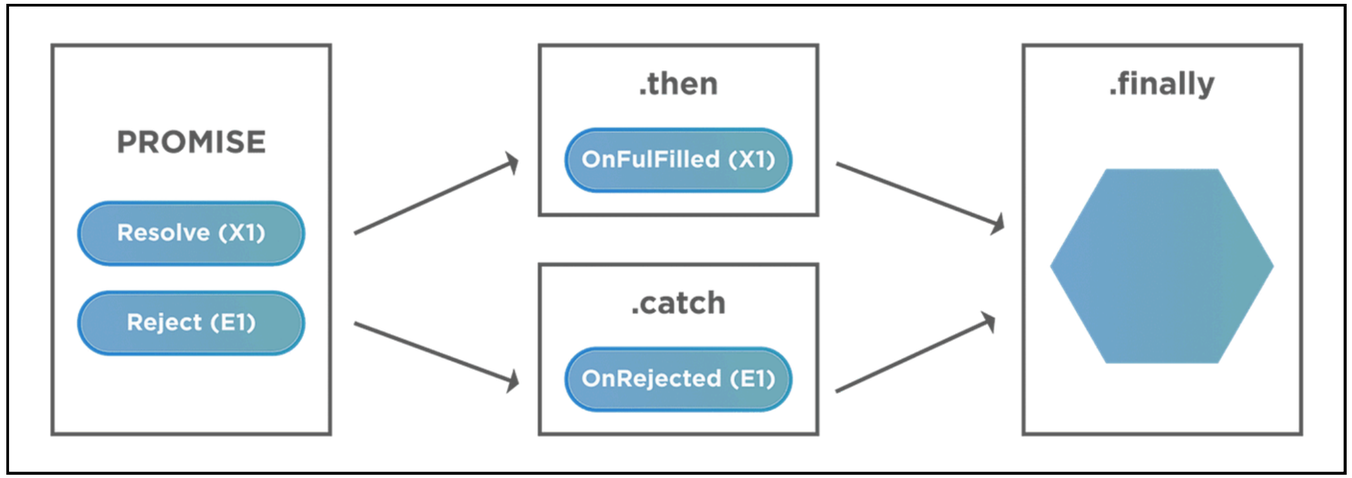
```
let promise = new Promise((resolve, reject) => {
  setTimeout(() => {
    resolve("Data loaded!");
  }, 2000);
});

promise.then(result => {
  console.log(result);
});
```



Promise in JavaScript

Promise Flow:



Promise = handle async result easier than callback = future result

- ***then() = success***
- ***catch() = error***

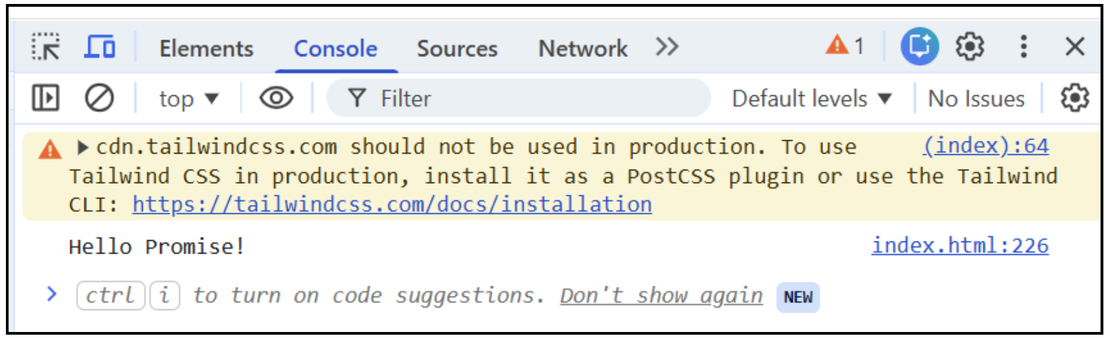
Promise + Async/AwaitAsync / Await

What is **async/await**?

- async → function returns a Promise
- await → wait for result

Cleaner way to use **Promise**

Example:




```
async function getData() {
  let promise = new Promise(resolve => {
    setTimeout(() => {
      resolve("Hello Promise!");
    }, 2000);
  });

  let result = await promise;
  console.log(result);
}

getData();
```

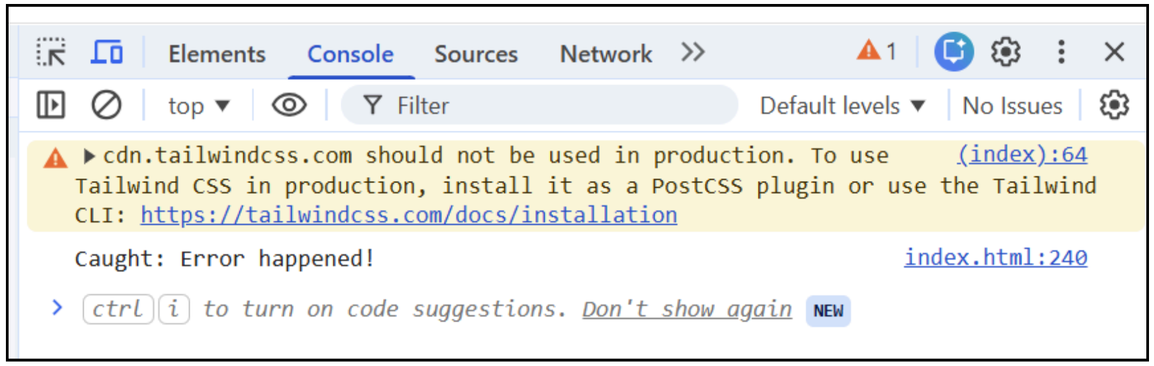
Promise + Async/Await

What is **async/await**?

- async → function returns a Promise
- await → wait for result

Cleaner way to use **Promise**

Example: Error Handling (Async/Await)




```
async function getData() {
  try {
    let promise = new Promise((resolve, reject) => {
      reject("Error happened!");
    });

    let result = await promise;
    console.log(result);
  } catch (error) {
    console.log("Caught:", error);
  }
}

getData();
```

Promise in JavaScript

Why Use Promise?

Before Promise → we use **callback**

Problem: Callback Hell **(hard to read)**

Promise makes async code:

- cleaner
- easier to manage

- Callback → basic
- Promise → better
- Async/Await → easiest

HTML DOM

DOM stands for **Document Object Model.**

When a web page is loaded, the browser creates a **DOM** of the page.

The DOM represents the HTML document as a **tree of objects.**

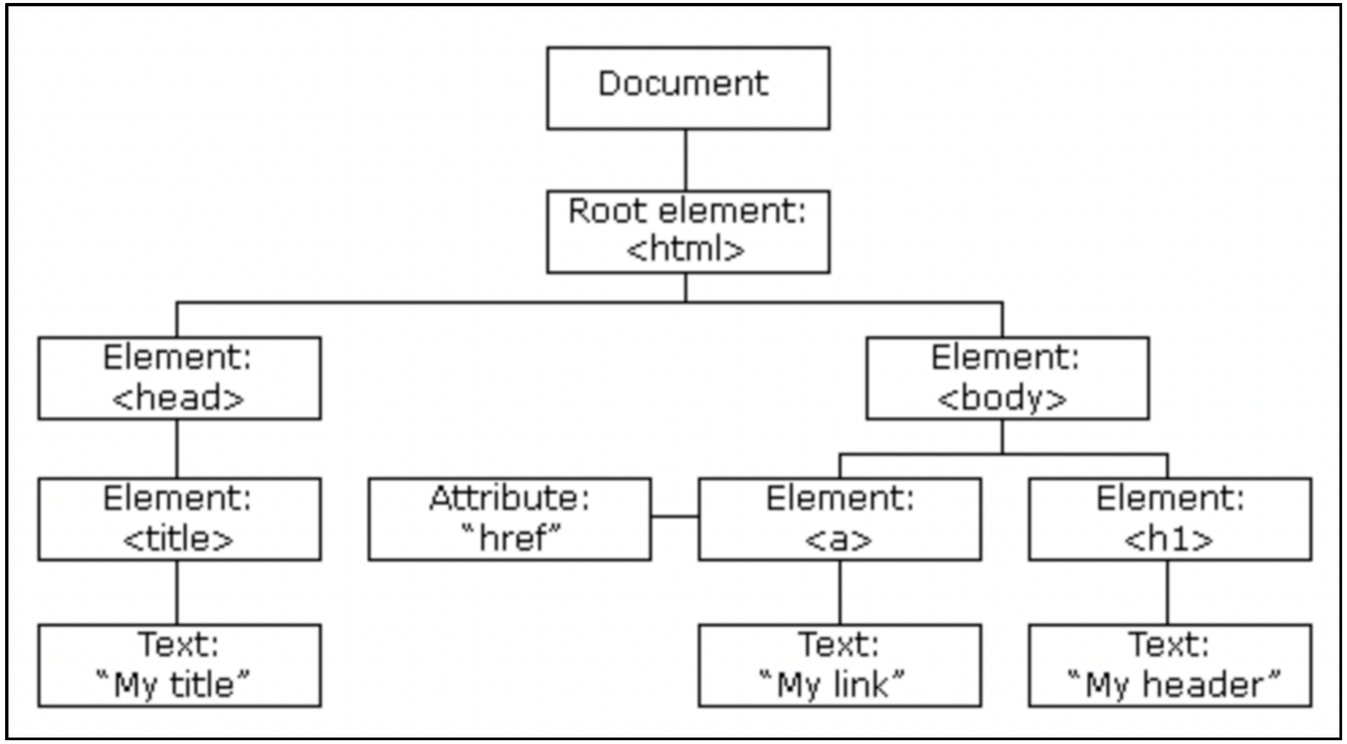
With JavaScript, we can use the **DOM** to:

- find HTML elements
- change content
- change styles
- change attributes
- create new elements
- remove elements
- respond to events

HTML DOM

The HTML DOM Tree of Objects

A web page is structured like a tree:



Important parts

- **Document** → the whole web page
- **Element** → HTML tags
- **Attribute** → extra information
- **Text** → the text inside elements

HTML DOM

DOM Methods and Properties

In the **DOM, HTML elements** are treated as **objects.**

Property — **A property** is a value you can get or set.

Example:

```
element.innerHTML = "Hello";
```

HTML DOM

DOM Methods and Properties

In the **DOM, HTML elements** are treated as **objects.**

Method — **A method** is an action you can perform.

Example:

```
document.getElementById("demo");
```

HTML DOM

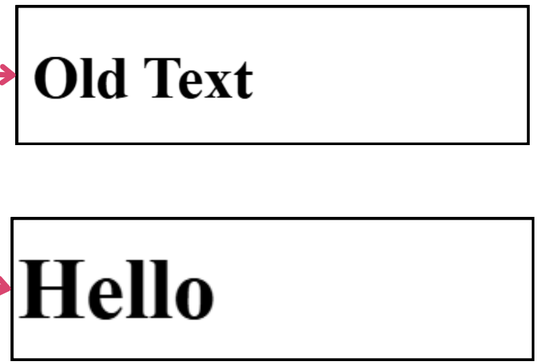
DOM Methods and Properties

In the **DOM, HTML elements** are treated as **objects.**

Example:

```
<p id="demo">Old Text</p>

<script>
  document.getElementById("demo").innerHTML = "Hello";
</script>
```



- getElementById() is a **method**
- innerHTML is a **property**

HTML DOM

The document Object

The document object represents the whole **HTML page.**

It is the **owner of all other objects** in the page.

That means JavaScript usually starts DOM access with:

```
document
```

Example:

```
document.getElementById("title");
document.createElement("p");
```

HTML DOM

Finding HTML Elements

JavaScript can find elements in different ways.

Method	Description
document.getElementById(id)	Finds an element by id
document.getElementsByTagName(name)	Finds elements by tag name
document.getElementsByClassName(name)	Finds elements by class name
document.querySelector(selector)	Finds the first matching CSS selector
document.querySelectorAll(selector)	Finds all matching CSS selectors

HTML DOM

Finding HTML Elements

JavaScript can find elements in different ways.

Example:

```
<h1 id="title">Welcome</h1>
<p class="text">Paragraph 1</p>
<p class="text">Paragraph 2</p>

<script>
  let title = document.getElementById("title");
  console.log(title);

  let paragraphs = document.getElementsByClassName("text");
  console.log(paragraphs);

  let firstParagraph = document.querySelector(".text");
  console.log(firstParagraph);
</script>
```



Welcome
Paragraph 1
Paragraph 2

HTML DOM

Changing HTML Elements

You can change the content, attributes, and style of elements.

1. **Change Content**

```
element.innerHTML = "New content";
element.textContent = "New content";
```

Example:

```
<p id="demo">Old content</p>

<script>
  document.getElementById("demo").textContent = "New content";
</script>
```



New content

HTML DOM

Changing HTML Elements

You can change the content, attributes, and style of elements.

2. **Change Attributes**

```
element.attribute = "new value";
element.setAttribute("attribute", "value");
```

Example:

```
<img id="photo" alt="Old image" src="old.jpg">

<script>
  let img = document.getElementById("photo");
  img.src = "new.jpg";
  img.setAttribute("alt", "New image");
</script>
```





HTML DOM

Changing HTML Elements

You can change the content, attributes, and style of elements.

3. **Change Style**

```
element.style.property = "value";
```

Example:

```
<p id="text">Hello DOM</p>

<script>
  let p = document.getElementById("text");
  p.style.color = "blue";
  p.style.fontSize = "24px";
</script>
```



Hello DOM

HTML DOM

Adding and Deleting Elements

JavaScript can create, add, replace, and remove elements.

Method	Description
document.createElement(element)	Creates a new element
parent.appendChild(element)	Adds an element
parent.removeChild(element)	Removes an element
parent.replaceChild(new, old)	Replaces an element
document.write(text)	Writes directly to the page output stream

HTML DOM

Adding and Deleting Elements

JavaScript can create, add, replace, and remove elements.

Example: Create and Add Element

```
<div id="box" class="border p-4 rounded"></div>

<script>
  let p = document.createElement("p");
  p.textContent = "This is a new paragraph";

  document.getElementById("box").appendChild(p);
</script>
```





HTML DOM

Adding and Deleting Elements

JavaScript can create, add, replace, and remove elements.

Example: Remove Element

```
<ul id="list">
  <li id="item1">Apple</li>
</ul>

<script>
  let parent = document.getElementById("list");
  let child = document.getElementById("item1");

  parent.removeChild(child);
</script>
```

HTML DOM

Adding and Deleting Elements

JavaScript can create, add, replace, and remove elements.

Example: Replace Element

```
<div id="container">
  <p id="oldText">Old paragraph</p>
</div>

<script>
  let newP = document.createElement("p");
  newP.textContent = "New paragraph";

  let container = document.getElementById("container");
  let oldP = document.getElementById("oldText");

  container.replaceChild(newP, oldP);
</script>
```



New paragraph

HTML DOM

Event Handlers

There are 2 simple ways to **handle events** (like clicking a button).

1. Inline Event (Inside HTML) = quick and simple
2. DOM Event (JavaScript) = professional and clean

JavaScript can respond to **user actions** such as:

- click
- mouseover
- keydown
- input
- submit

HTML DOM

Event Handlers

There are 2 simple ways to **handle events** (like clicking a button).

1. **Inline Event** (Inside HTML) — put JavaScript **directly inside HTML**
    - **Example:**

```
<button onclick="showMessage()">Click Me</button>

<script>
  function showMessage() {
    console.log("Button clicked");
  }
</script>
```

How it works:

1. User clicks button
2. HTML calls function showMessage()
3. Function runs

HTML DOM

Event Handlers

There are 2 simple ways to **handle events** (like clicking a button).

2. **DOM Event** (JavaScript)— use JavaScript to attach event to element
    - **Example:**

```
<button id="btn">Click Me</button>

<script>
  document.getElementById("btn").onclick = function () {
    console.log("Button clicked");
  };
</script>
```

How it works:

1. JavaScript finds button (id="btn")
2. Adds click event
3. When user clicks → function runs

HTML DOM

Event Handlers

There are 2 simple ways to **handle events** (like clicking a button).

Visual Comparison:

```
<button onclick="doSomething()">
```

Inline

```
element.onclick = function () { }
```

DOM

```
element.addEventListener("click", function () {
  console.log("Clicked");
});
```

Best Method - Advanced

HTML DOM

addEventListener()

The **addEventListener()** method attaches an event handler to an element.

Advantages

- does not overwrite existing handlers
- can attach many handlers to one element
- cleaner and more flexible

HTML DOM

addEventListener()

Syntax

```
element.addEventListener(event, function, useCapture);
```

Usually, useCapture is omitted:

```
element.addEventListener("click", function () {
  console.log("Clicked");
});
```

HTML DOM

addEventListener()

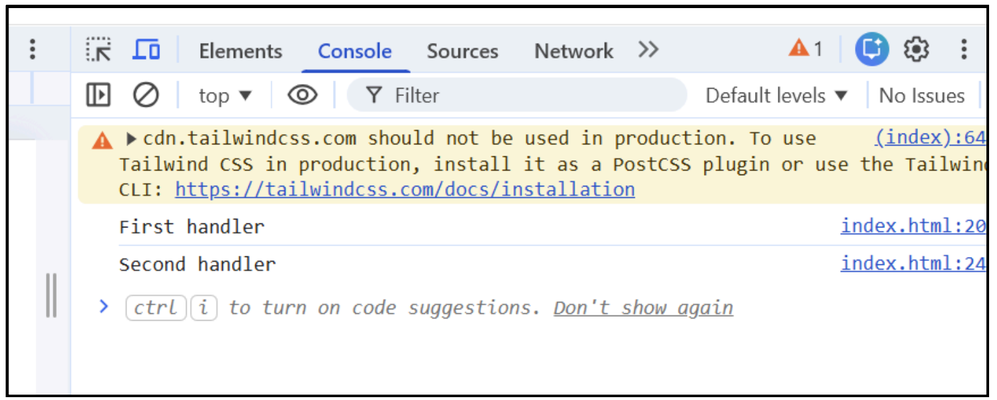
Example

```
<button id="myBtn" class="px-4 py-2 border rounded-full">Click Me</button>

<script>
  let btn = document.getElementById("myBtn");

  btn.addEventListener("click", function () {
    console.log("First handler");
  });

  btn.addEventListener("click", function () {
    console.log("Second handler");
  });
</script>
```



HTML DOM

innerHTML Security Warning

Unsafe

```
element.innerHTML = userInput;
```

Why unsafe?
Because if userInput contains HTML or JavaScript code, it may create an XSS vulnerability.

What is XSS Vulnerability?
XSS (Cross-Site Scripting) is a security problem where an attacker can **inject malicious JavaScript code into a website.**

XSS = “Hacker can run their script inside your website” = Cross-Site Scripting attack

HTML DOM

innerHTML Security Warning

How XSS Happens

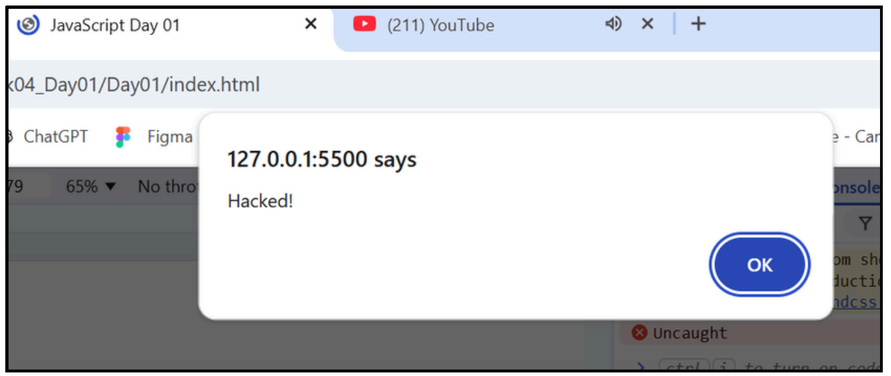
It usually happens when:

If userInput contains HTML **or JavaScript,** it will be executed.

Example of XSS Attack

```
let userInput = "<script>alert('Hacked!')</script>";

document.getElementById("demo").innerHTML = userInput;
```



This means attacker code is running on your page

HTML DOM

innerHTML Security Warning

How XSS Happens

It usually happens when:

If userInput contains HTML **or JavaScript,** it will be executed.

Why is XSS Dangerous?

An attacker can:

- steal user data (cookies, passwords)
- modify your website content
- redirect users to fake websites
- perform actions as the user

HTML DOM

innerHTML Security Warning

Safe

```
element.textContent = userInput;
```

Why safer?
Because it treats the input as **plain text**, not HTML.

Example

```
let userInput = "<script>alert('Hacked!')</script>";

document.getElementById("demo").textContent = userInput;
```

No script runs

HTML DOM

innerHTML Security Warning

- innerHTML → allows **HTML + script execution**
- textContent → shows only **text (safe)**

Or

- innerHTML = opening a file (can run virus)
- textContent = reading text only

Use textContent instead of innerHTML

Browser BOM

What is BOM?

The **Browser Object Model (BOM)** allows JavaScript to **interact with the browser.**

It lets JavaScript:

- control the browser window
- access browser information
- navigate between pages

DOM → control web page
BOM → control browser

Browser BOM

The window Object

The window object represents the **browser window**

It is the **top-level object**

Example:

```
console.log(window.innerWidth);
console.log(window.innerHeight);
```

Common Methods

```
window.open("https://google.com"); // open new tab
window.close(); // close window
window.moveTo(100, 100); // move window
window.resizeTo(800, 600); // resize window
```

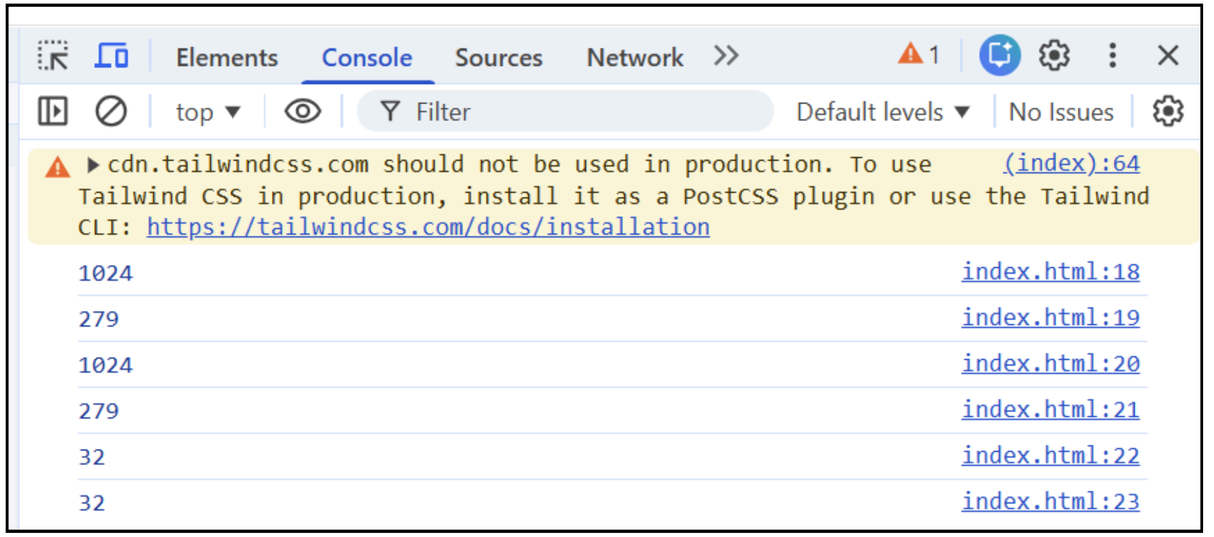
Browser BOM

The Screen Object

The screen object rcontains information about the user's screen

Example:

```
console.log(screen.width);
console.log(screen.height);
console.log(screen.availWidth);
console.log(screen.availHeight);
console.log(screen.colorDepth);
console.log(screen.pixelDepth);
```



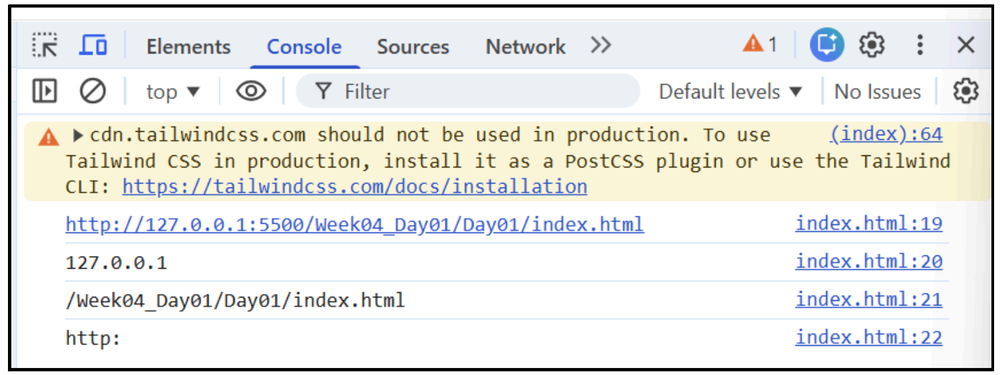
Browser BOM

The Location Object

The location object contains information about the current URL

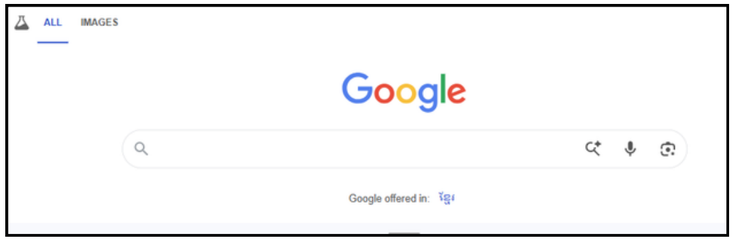
Example:

```
console.log(location.href);      // full URL
console.log(location.hostname);  // domain
console.log(location.pathname);  // path
console.log(location.protocol);  // http / https
```



Redirect Page

```
location.assign("https://google.com");
```



Browser BOM

The History Object

The history object allows navigation through browser history

Example:

```
history.back();    // go back
history.forward(); // go forward
```

Same as clicking browser buttons

Browser BOM

Cookies

What are Cookies?

Cookies store small pieces of data in the browser

Used to remember:

- user name
- login info
- preferences

Browser BOM

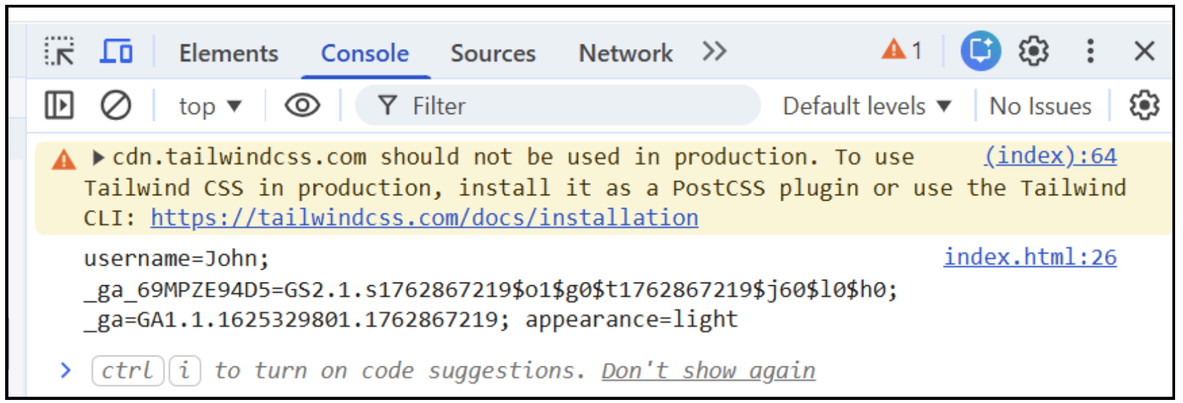
Cookies

1. Create Cookie

```
document.cookie = "username=John; expires=Thu, 18 Dec 2030 12:00:00 UTC";
```

2. Read Cookie

```
console.log(document.cookie);
```



3. Update Cookie

```
document.cookie = "username=John Smith; expires=Thu, 18 Dec 2030 12:00:00 UTC; path=/";
```

Browser BOM

Important Notes

window is optional

```
window.location.href
location.href // same
```

- window = browser
- screen = user screen
- location = URL
- history = navigation
- navigator = browser info
- cookie = stored data

Some methods may not work due to browser security

Example: window.close() **only works for windows opened by script**





THANK YOU

For your attention !

